ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

Параллельные вычисления

ЛАБОРАТОРНАЯ РАБОТА № 1

Тема: Методика измерения времени выполнения заданного фрагмента программы

Группа: БПМ-22-ПО-1

Студент: Нургалиев Родион Дамирович

Ссылка на репозиторий с кодом:
https://github.com/LLrodyaLL/parallel_computing_misis_nurgaliev_r_d_bpm_22_po_1/tree/main/lab01

2026

1. Цель работы

Целью данной лабораторной работы является изучение методов профилирования программ путём измерения длительности выполнения заданных фрагментов кода. В ходе работы рассматриваются три способа измерения времени: с помощью функции GetTickCount64(), с помощью функций QueryPerformanceCounter() / QueryPerformanceFrequency() (QPC), а также с помощью аппаратного счётчика TSC (RDTSC).

2. Описание задания

Вариант задания: умножение двух квадратных матриц размером 500×500 элементов типа double, заполненных случайными числами в диапазоне [0; 10]. Реализован тройной вложенный цикл i-j-k. Число замеров: K = 30. Перед серией замеров производится один прогревочный запуск. Поток привязан к ядру №0 функцией SetThreadAffinityMask(), приоритет процесса повышен до HIGH_PRIORITY_CLASS.

Профилируемый фрагмент кода (функция multiplyMatrices):

```
for (size_t i = 0; i < n; ++i) {
    for (size_t j = 0; j < n; ++j) {
        double sum = 0.0;
        for (size_t k = 0; k < n; ++k)
            sum += A[i][k] * B[k][j];
        C[i][j] = sum;
    }
}
```

3. Листинг программы

3.1. main.cpp

```
#include "matrix.hpp"
#include "timers.hpp"
#include "csv_writer.hpp"
#include <windows.h>
#include <iostream>
#include <vector>
#include <filesystem>

int main() {
    const std::size_t N = 500;
    const int K = 30;
    SetPriorityClass(GetCurrentProcess(), HIGH_PRIORITY_CLASS);
    SetThreadAffinityMask(GetCurrentThread(), 1);

    Matrix A = createMatrix(N), B = createMatrix(N), C = createMatrix(N);
    fillRandom(A, 42); fillRandom(B, 123);

    // Прогревочный запуск
    multiplyMatrices(A, B, C); zeroMatrix(C);

    std::vector<MeasurementRow> rows;
    for (int run = 1; run <= K; ++run) {
        zeroMatrix(C);
        double gettickMs  = measureWithGetTickCount64(A, B, C);
        zeroMatrix(C);
```

```
        double qpcMs      = measureWithQPC(A, B, C);
        zeroMatrix(C);
        uint64_t rdtscTicks = measureWithRDTSC(A, B, C);
        rows.push_back({run, gettickMs, qpcMs, rdtscTicks});
    }
    writeMeasurementsToCsv("results/raw_measurements.csv", rows);
}
```

3.2. timers.cpp

```
#include "timers.hpp"
#include <windows.h>
#include <intrin.h>

double measureWithGetTickCount64(const Matrix& A, const Matrix& B, Matrix& C) {
    ULONGLONG start = GetTickCount64();
    multiplyMatrices(A, B, C);
    ULONGLONG end = GetTickCount64();
    return static_cast<double>(end - start);
}

double measureWithQPC(const Matrix& A, const Matrix& B, Matrix& C) {
    LARGE_INTEGER freq, start, end;
    QueryPerformanceFrequency(&freq);
    QueryPerformanceCounter(&start);
    multiplyMatrices(A, B, C);
    QueryPerformanceCounter(&end);
    return (double)(end.QuadPart - start.QuadPart) * 1000.0 / freq.QuadPart;
}

uint64_t measureWithRDTSC(const Matrix& A, const Matrix& B, Matrix& C) {
    unsigned __int64 start = __rdtsc();
    multiplyMatrices(A, B, C);
    unsigned __int64 end   = __rdtsc();
    return static_cast<uint64_t>(end - start);
}
```

4. Результаты измерений

4.1. Таблица сырых измерений (K = 30 замеров)

№ замера	GetTickCount64, мс	QPC, мс	RDTSC, тактов
1	141	129.7253	382 277 466
2	156	135.1709	392 604 240
3	141	133.1408	386 052 971
4	125	133.8070	383 936 350
5	141	130.8130	382 709 986
6	125	135.4571	426 862 545
7	156	134.7383	384 720 107
8	140	134.4003	390 293 817
9	141	134.2176	395 049 262
10	141	135.3257	388 642 790
11	141	133.4617	385 952 908

12	125	130.3867	389 523 194
13	141	132.3961	383 865 910
14	140	132.2268	378 651 881
15	141	130.7330	380 759 963
16	125	132.0160	383 410 167
17	141	139.1187	395 918 512
18	141	136.3335	397 368 233
19	141	134.6526	408 365 011
20	141	134.7707	401 214 182
21	125	133.2351	385 012 044
22	141	133.1608	385 757 789
23	125	134.2646	435 154 865
24	141	134.2184	391 182 320
25	125	133.3918	384 823 570
26	140	132.6443	385 452 945
27	125	131.8162	382 175 388
28	141	130.2092	383 431 782
29	125	131.0569	378 235 261
30	125	130.2279	386 830 466

4.2. Статистическая обработка результатов

Для каждого метода вычислены: минимальное время t_min, среднее арифметическое t_avg(T) по всей выборке, а также среднее t_avg(T') после отбрасывания выбросов по критерию 3σ. Доверительный интервал рассчитан при p = 0,95 по формуле $\Delta = 1{,}96 \cdot \sigma / \sqrt{K'}$.

Метод	t_min	t_avg (T)	t_avg (T')	Дов. интервал (p=0.95)
GetTickCount64	125.000	136.567	136.567	±3.216 мс
QPC	129.725	133.237	133.237	±0.744 мс
RDTSC (тактов)	378235261	390541198	389002795	±3554517 тактов

4.3. Гистограммы распределения измерений

Рисунок 1 — Гистограмма измерений GetTickCount64

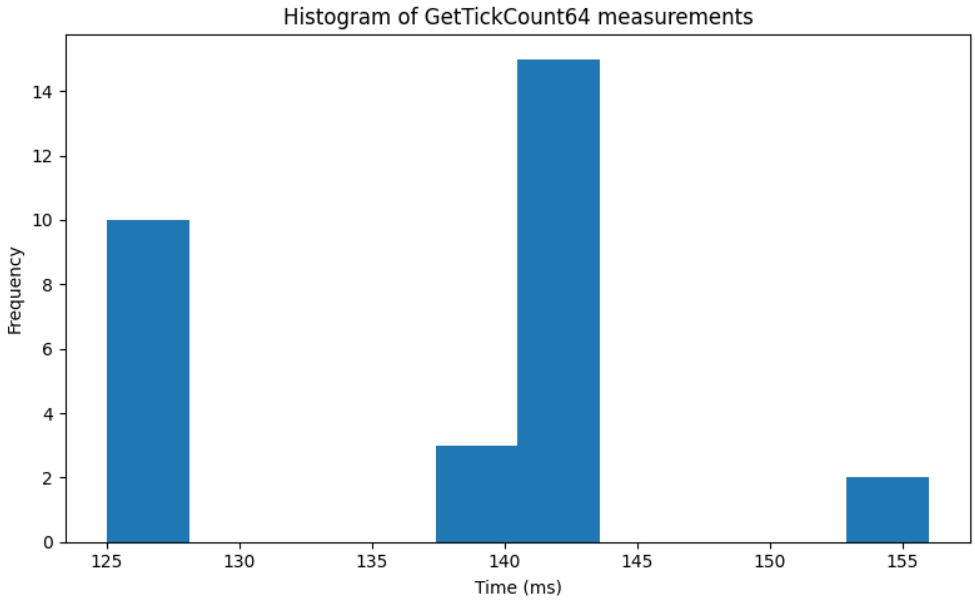


Рисунок 2 — Гистограмма измерений QueryPerformanceCounter (QPC)

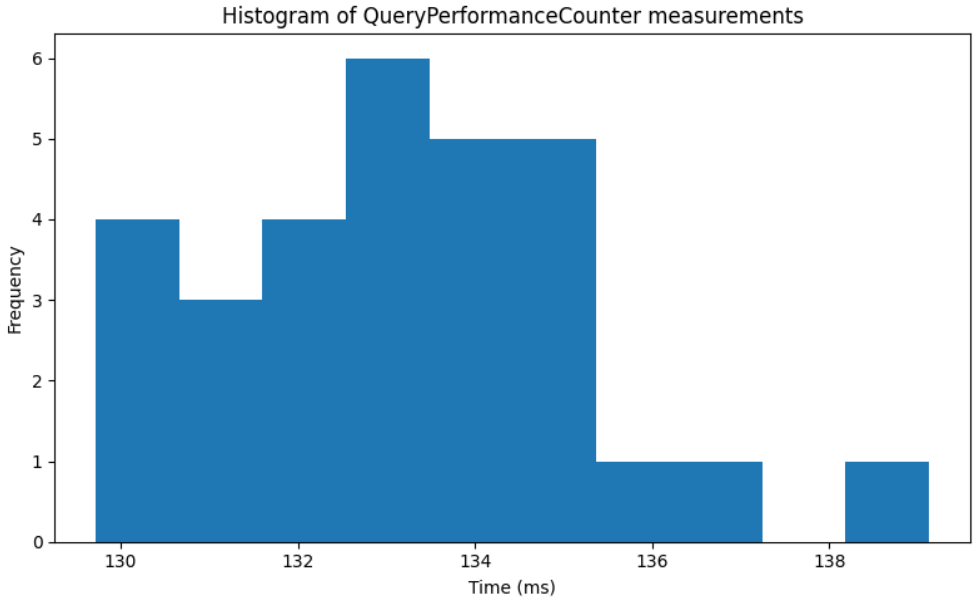
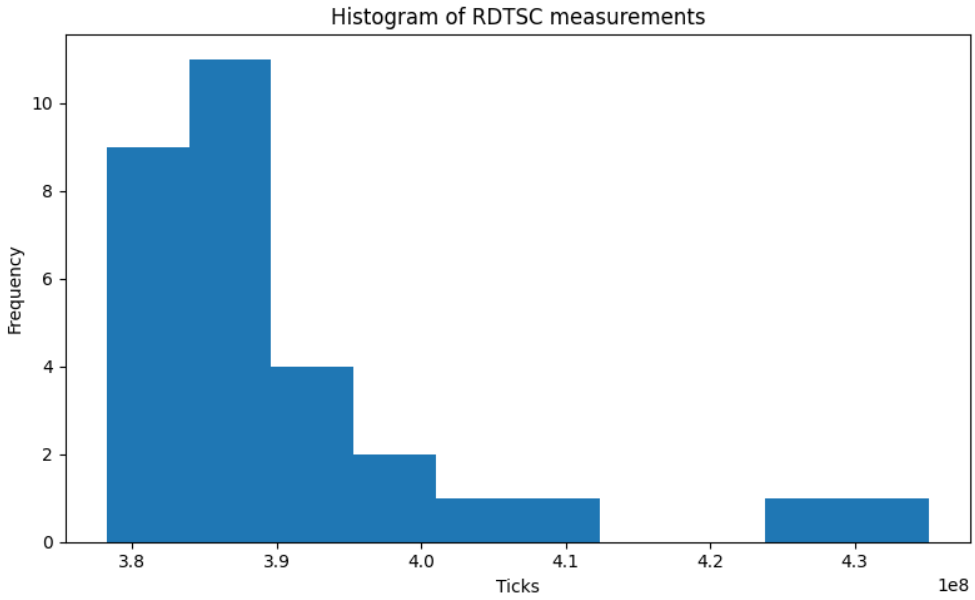


Рисунок 3 — Гистограмма измерений RDTSC



5. Выводы

В ходе выполнения лабораторной работы были изучены три метода измерения времени выполнения программного фрагмента и применены к задаче умножения квадратных матриц 500×500.

1. GetTickCount64 - показал дискретные значения (125, 140, 141, 156 мс), обусловленные разрешением системного таймера (~15,6 мс). Метод непригоден для измерений с высокой точностью. Минимальное измеренное время: 125 мс, среднее (после отброса выбросов): 136.6 ± 3.2 мс.

2. QueryPerformanceCounter (QPC) - продемонстрировал высокую точность и стабильность. Минимальное время: 129.725 мс, среднее (после отброса выбросов): 133.237 ± 0.744 мс. Метод является рекомендуемым для измерений средних и больших интервалов.

3. RDTSC - наиболее высокоточный метод на аппаратном уровне. Минимальное значение: 378 235 261 тактов, среднее (после отброса выбросов): 389 002 795 ± 3 554 517 тактов. Наблюдались единичные выбросы (замеры 6 и 23), вероятно вызванные прерываниями ОС. После их отброса по критерию 3σ стабильность результатов существенно возросла.

Привязка потока к одному ядру (SetThreadAffinityMask) и повышение приоритета процесса (HIGH_PRIORITY_CLASS) позволили снизить влияние планировщика ОС. Прогревочный запуск обеспечил стабилизацию кэша, что подтверждается близостью результатов QPC и RDTSC. Рекомендуется использовать QPC для измерения времени в миллисекундах и RDTSC — для тонкого профилирования на уровне тактов.

Код и результаты работы размещены в открытом репозитории GitHub по ссылке ниже:
https://github.com/LLrodyaLL/parallel_computing_misis_nurgaliev_r_d_bpm_22_po_1/tree/main/lab01